

Project

Summarizing Cars for Sale on DoneDeal



Project

Summarizing Cars for Sale on DoneDeal

by

Jack Curtin
L00179658



[GitHub.com/JackCurtin/Summarizing-Doneddeal-Project](https://github.com/JackCurtin/Summarizing-Doneddeal-Project)

Supervisor:	Pauric Dawson
Course:	Contemporary Software Development 2023/2024
Project Duration:	March, 2024 - May, 2024
Faculty:	Atlantic Technology University



Decleration

I hereby certify that the material, which I now submit for assessment on the programmes of study leading to the award of Bachelor of Science in Applied Computing, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my own work. No portion of the work contained in this thesis has been submitted in support of an application for another degree or qualification to this or any other institution. I understand that it is my responsibility to ensure that I have adhered to ATU's rules and regulations.

I hereby certify that the material on which I have relied on for the purpose of my assessment is not deemed as personal data under the GDPR Regulations. Personal data is any data from living people that can be identified. Any personal data used for the purpose of my assessment has been pseudonymised and the data set and identifiers are not held by ATU. Alternatively, personal data has been anonymised in line with the Data Protection Commissioners Guidelines on Anonymisation.

I consent that my work will be held for the purposes of education assistance to future students and will be shared on the ATU Computing website (<http://www.atucomputingdonegal.com/>) and Research THEA website (<https://research.thea.ie/>). I understand that documents once uploaded onto the website can be viewed throughout the world and not just in Ireland. Consent can be withdrawn for the publishing of material online by emailing Thomas Dowling; Head of Department at jade.lyons@atu.ie to remove items from the ATU Computing website and by email emailing Denise McCaul; Systems Librarian at denise.mccaul@atu.ie to remove items from the Research THEA website. Material will continue to appear in printed formats once published and as websites are public medium, ATU cannot guarantee that the material has not been saved or downloaded.

Signature of Candidate: **Jack Curtin**

Date: **22/05/2024**

Abstract

This report showcases the process of creating an application which assists used car buyers in finding a car for a fair price. This application will be created using .Net Maui, which involves both C# and XAML coding files. The data will be coming from DoneDeal, which is an Irish based website consumers use to purchase both new and second hand vehicles.

Abbreviations

Abbreviation	Definition	Page
.NET MAUI	.NET Multi-platform App UI	4
FR	Functional Requirement	4
NFR	Non-Functional Requirement	5
CI/CD	Continuous Integration and Continuous Delivery	10
MVVM	Model-View-ViewModel	10
IDE	Integrated Development Environment	10

Contents

Declaration	i
Abstract	ii
Abbreviations	iii
1 Introduction	1
1.1 Background	1
1.2 Aims and Objective	2
1.3 Report Outline	2
2 Design	4
2.1 System Requirements	4
2.1.1 Functional	4
2.1.2 Non-Functional	5
2.1.3 User Stories	5
2.2 System Diagrams	7
2.2.1 Use Case Diagram	7
2.2.2 Flow Chart	7
2.3 System Tools & Hardware Requirements	8
3 Implementation	10
3.1 Initializing GitHub	10
3.1.1 Continuous Integration and Continuous Delivery	10
3.2 Configuring the Solution	12
3.2.1 Packages	12
3.3 Code Structure	13
3.3.1 Models	13
3.3.2 Views	14
3.3.3 ViewModels	17
3.3.4 Services	19
4 Testing	23
4.1 Unit Tests	23
4.1.1 Services Unit Tests	24
5 Conclusion	27
5.1 Challenges	27
5.2 Learning Outcomes	28
References	29

List of Figures

1.1	Average Asking Price for Cars from Feb 2011 to Feb 2024 (AutoTrader PLC 2024) . .	2
2.1	Use Case Diagram: Summarizing Cars for Sale	7
2.2	Flow Chart of Artifact	8
3.1	dotnet.yml GitHub Actions workflow	11
3.2	MVVM Structure	13
3.3	MainPage of Artifact	14
3.4	SummaryPage of Artifact	16
4.1	Test Explorer Output of a Basic Unit Test	24
4.2	Test Explorer Output for the Artifact's C# Code	24

List of Tables

2.1	Functional Requirements	4
2.2	Non-Functional Requirements	5
2.3	US1: User Story 1	6
2.4	US2: User Story 2	6
2.5	US3: User Story 3	6
4.1	CarMakes - GetCarMakesAsync_Fail	25
4.2	CarMakes - GetCarMakesAsync_Success	25
4.3	CarMakes - GetCarMakesAsync_InvalidURL	25
4.4	GetCarModelsAsync_Fail	26

Listings

3.1	Main YAML file: <code>dotnet.yml</code>	11
3.2	XAML Drop Down	15
3.3	XAML View Car Details Button	15
3.4	XAML Average Mileage	17
3.5	XAML Highlighting Specific Car Advertisements	17
3.6	MainViewModel Constructor	18
3.7	SummaryViewModel Constructor	19
3.8	CarMakeService Extracting Car Makes	20
3.9	CarModelService Extracting Car Models	21
3.10	CarDetailService Pagination	21
4.1	Example of a Basic Unit Test	23

1. Introduction

Buying a car in Ireland has slowly transitioned from being a luxury to a necessity. As such, making sure you pay the right price for a car has become more relevant than ever. Nowadays consumers can easily find advertisements for cars based on make and model using websites like DoneDeal, Carzone.ie, carsireland.ie etc. Whilst these sites are great for viewing the details of a specific advert, they make it difficult to calculate the average cost of the exact make/model/year of car the consumer is considering. Given the rising cost of used cars it has become crucial for consumers to be aware of what a “fair price” is for the car they are hoping on purchasing.

This goal of this project is to help consumers find out if they are paying a fair price for the car they are interested in. The inspiration for this project stemmed from personal experience of looking for a car on the DoneDeal site - which was a 2016 3-series BMW. The process of researching available options showcased just how much variance there was in terms of the advertised price and mileage of each advertisement. This made the process of trying to determine if a particular advertisement was a good deal or not rather difficult, and ultimately led to wishing there was more information available. Having access to information such as the average price and mileage of a specific car would allow a potential buyer to have more insight to make an informed decision. Thus, an application which can display average cost and mileage information will increase the likelihood of a buyer securing a fairer deal.

1.1. Background

Many Irish news outlets have covered this topic since it began seemingly around the time of the Covid-19 Pandemic, where car sales in *“Europe dropped by over 65 percent between March and April 2020”* (Statista 2020). An Irish Examiner article from February 2024 highlighted that used car prices in March 2024 were up 85% when being compared to the same time in 2020 (Irish Examiner 2024). Even in July 2023, the Irish Independent found that *“used-car prices are still 79% higher than pre-pandemic levels”* (Independent.ie 2023).

AutoTrader Group, a UK based automotive marketplace, has a fantastic visual showing the average asking price for all car segments sold on their site. **Figure 1.1** from AutoTrader visualizes the average asking price for cars from February 2011 until February 2024. This graph very closely resembles what the Irish market has been doing, and what the Irish media have been saying about the used car market. DoneDeal made a blog post discussing how the price growth for cars in 2023 eased when being compared to 2020 - 2022. They noted that the *“lingering effect of the pandemic microchip shortage, coupled with Brexit and then the outbreak of war in Ukraine caused significant disruption to both the supply of, and the demand for, used cars”* (DoneDeal 2024).

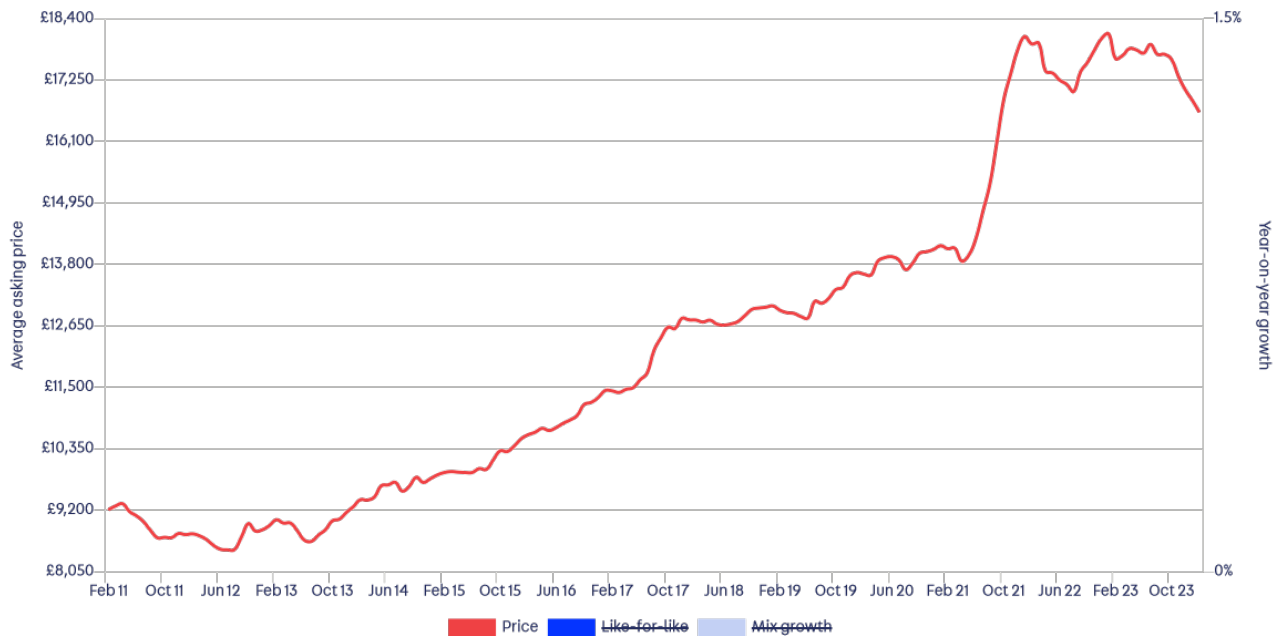


Figure 1.1: Average Asking Price for Cars from Feb 2011 to Feb 2024 (AutoTrader PLC 2024)

1.2. Aims and Objective

The aim of this project is to create a .NET Maui application which will allow consumers to find information on a given car based on make/model/year. This application will be able to get the car advertisement information from DoneDeal, and will store and transform the data into a usable format.

To achieve this aim, the main objectives of this project will be as follows:

1. Use C# to obtain car advertisement information from DoneDeal.
2. Compile this data into a structured format allowing for ease of integration.
3. Create a XAML pages which will allow the user to view this information.
4. Expand my knowledge of software development and DevOps.

The result will be a fully functional application which will allow users to easily find information such as average cost and average mileage for the car they are interested in buying. This will allow the consumer to make a more informed buying decision, and ultimately be more confident that they are buying a vehicle at a fair price.

1.3. Report Outline

This report will be structured as follows:

1. Introduction - Provides some background into the inspiration of this project and gives an outline of the aims and objectives of this project.

-
2. Design - Gives an overview on the tools which will be used to complete this project whilst giving a blueprint on the overall design of the application.
 3. Implementation - Goes into detail on the code required to complete this project.
 4. Testing - Showcases an array of tests which were completed to ensure this project functions as intended.
 5. Conclusion - Discusses any shortcomings on the project and provides suggestions on how the scope could expand for this to become a full-fledged application.

2. Design

In order to create a successful artifact, it is essential to first plan and design how the solution should function. For this reason, before beginning development work, this report will consider the system requirements, which includes functional and non functional requirements. User stories “*have been widely accepted as artifacts to capture the user requirements in agile software development*” (Raharjana, Siahaan, and Fatichah 2021), and as such they shall be used to determine how the artifact should function.

The system diagrams will take inspiration from these system requirements. Unified Modeling Language (UML) diagrams will be used to visualize how the artifact stakeholders will interact with the system. The first of these diagrams will be a use case diagram depicting how the artifacts users will use the system. This will be followed by a flow diagram which will break down the overall workflow process of the artifact.

The tools used to create the the artifact will be the coding language C# in conjunction with XAML pages. This framework of tools is referred to as a .NET Multi-platform App UI (.NET MAUI) application. In addition to having a software which can run a .NET MAUI application, the user should also have a computer which should at a minimum be able to run the artifact.

2.1. System Requirements

2.1.1. Functional

Requirement ID	Requirement Statement
FR01	The application shall have a home screen displaying the purpose of the project.
FR02	The application shall have a screen showing a list of the available car makes.
FR03	The application shall display a list of car models based on the car make the user has selected.
FR04	The application shall load and store the car information in a database based on the user inputs.
FR05	The application shall organize the car information which it has downloaded in a structured manner.
FR06	The application shall display the loaded data on a screen.

Table 2.1: Functional Requirements

Functional requirements (FRs) are defined in software development as requirements which “*capture the intended behavior of the system*” (Malan, Bredemeyer, et al. 2001). As such, the requirements of this artifact are listed with a numeric Requirement ID and a subsequent Requirement Statement, which can be seen in **Table 2.1**. These requirements will be worded as “X shall do/have/display”, as each requirement will relate to how the artifact should function.

2.1.2. Non-Functional

In software development, non-functional requirements (NFRs) “*relate to system properties which do not directly affect the features provided by the system*”(Jarzębowicz and Weichbroth 2021). These NFRs are listed with a numeric Functional ID, a Quality and a Requirement Statement. The Quality column will relate to the specific NFR area which the Requirement Statement relates to. The NFRs for this artifact can be seen in **Table 2.2** below.

Requirement ID	Quality	Requirement Statement
NFR01	Usability	The application should have a minimalist design accompanied with an easy to use feel.
NFR02	Performance	The application should not take longer than 30 seconds to get and store the car data requested by the user.
NFR03	Reliability	The application should run without failure.
NFR04	Performance	The application should be able to run without the user needing to download and install any additional packages.

Table 2.2: Non-Functional Requirements

2.1.3. User Stories

Perhaps the best method to determine the system requirements is with the aid of user stories. As mentioned, user stories are widely used in agile software development to determine the functionality that the user wishes to have in the artifact (Raharjana, Siahaan, and Fatichah 2021). User stories are often constructed in simple sentences as follows; “*As a [persona], I [want to], [so that]*” (Atlassian 2024). As such, the user stories for this report will follow a similar format, each with a unique ID and with the addition of an acceptance criteria area. **Table 2.3**, **Table 2.4** and **Table 2.5** showcase user stories which are in the perspective of the users of the artifact.

User Story 1	
As a user	I want to see the average mileage of a particular car,
So that I can make a more informed purchasing decision.	
Acceptance Criteria	
AC1. The artifact should have a user friendly interface. AC2. The artifact should give the user to option to select multiple car manufacturers. AC3. The artifact should have a screen to show the average mileage of the specific car the user has selected.	

Table 2.3: US1: User Story 1

User Story 2	
As a user,	I want to see the car makes available,
So that it is easier to find specific models based on the selected car manufacturer.	
Acceptance Criteria	
AC1. The artifact should have a drop-down for car manufacturer. AC2. Upon selecting a car manufacturer, the artifact should show the relevant car models from that manufacturer. AC3. The artifact should allow the user to select one of these car models from a drop-down.	

Table 2.4: US2: User Story 2

User Story 3	
As a user,	I want to filter by engine size,
So that I can fine tune my criteria.	
Acceptance Criteria	
AC1. Once the user has selected a car make and model they should see the available engine sizes. AC2. The artifact should allow the user to select one of these engine sizes from a drop down. AC2. The artifact should display the metrics of the cars based on the chosen engine size.	

Table 2.5: US3: User Story 3

2.2. System Diagrams

2.2.1. Use Case Diagram

Use case diagrams can be used to graphically describe how a software solution interacts with its users and vice-versa. **Figure 2.1** visualizes the behaviour of the artifact by showing how the users of the artifact will interact with the system. This figure is of a use case UML diagram which was created using Visio.

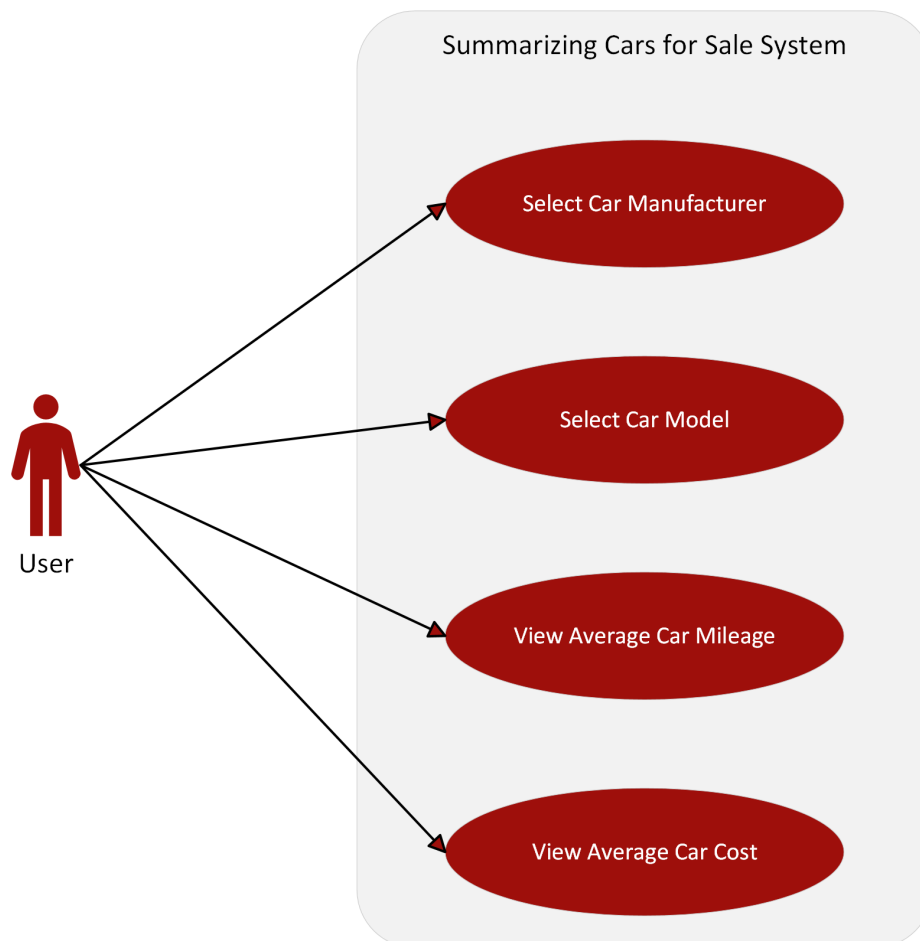


Figure 2.1: Use Case Diagram: Summarizing Cars for Sale

2.2.2. Flow Chart

Flow charts visually describe the steps involved in a process. These charts have proven to be very effective for simplifying complex workflows, particularly for non technical individuals (Xinogalos 2013). The flow chart in **Figure 2.2** visualizes the workflow of this artifact.

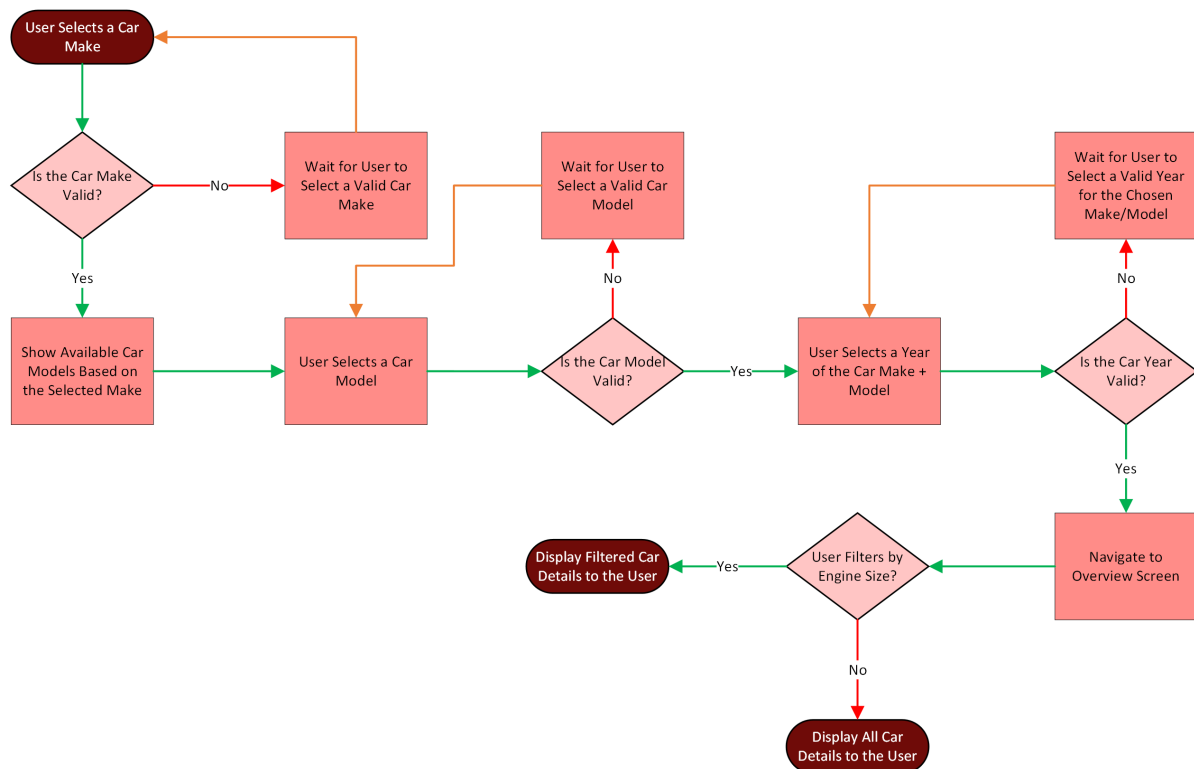


Figure 2.2: Flow Chart of Artifact

2.3. System Tools & Hardware Requirements

This artifact will be created as a .Net MAUI application, which is “a cross-platform framework for creating native mobile and desktop apps with C# and XAML” (Microsoft 2023). Visual Studio 2022 will be used as the Integrated Development Environment (IDE) for this application. To ensure the creation of this artifact is properly handled, it will be stored on GitHub. GitHub “is a cloud-based platform where you can store, share, and work together with others to write code” (GitHub 2024a). An additional benefit of using GitHub to store the artifact is that it allows for DevOps integration through GitHub Actions. DevOps, or Development and IT Operations, consists of “a model for delivering software as a service that comprises a set of practices and tools combining the responsibilities for both developing and operating the software” (Gall and Pigni 2022).

In terms of hardware requirements, the minimum requirements required to run the chosen IDE is listed below:

- ARM64 or x64 Quad-core processor.
- 4 GB of RAM.
- 20-50 GB of free space.

However to run this artifact, and to ensure a smooth experience, the following hardware requirements are recommended:

- ARM64 or x64 Quad-core processor with a capable speed of 3 GHz.
- 16 GB of RAM.
- 80 GB of free space.

3. Implementation

Now that the design of the artifact has been determined, the process of creating the artifact can be explored. Elements from the design chapter will act as a basis for the implementation of this artifact. To begin, this chapter will focus on how the GitHub repository was initialized, which is where the artifact will be stored and where subsequent versions of the artifact tracked. An additional benefit of using GitHub is the ability to integrate GitHub Actions, which allows for Continuous Integration and Continuous Delivery (CI/CD).

With the GitHub repository initialized, configuring the solution is the next step. This involves setting up the Integrated Development Environment (IDE), in this case Visual Studio 2022, which shall be used to create the artifact. It is also necessary to download and install any .NET Maui packages which will be needed to develop the artifact.

As this artifact is a .NET Maui application, the code structure will consist of both XAML and C# code. It will follow the Model-View-ViewModel (MVVM) structure, and as such the code will be organized in designated folders to allow for consistency, and to make it easier to manage.

3.1. Initializing GitHub

As mentioned, GitHub is a version control platform which shall be used to store and manage the artifact. The GitHub repository for this artifact, <https://github.com/JackCurtn/summarizing-donedeal-project>, was created as a private repository and with a security rule which ensures commits to the master branch must come from pull requests. This decision was made to keep the master branch as clean as possible, and to allow for further features to be incorporated with the GitHub repository.

3.1.1. Continuous Integration and Continuous Delivery

An advantage of using GitHub is the ability to easily integrate GitHub Actions, which can be used to automate the workflow. The process of automating workflows is more often referred to as CI/CD. CI/CD is used to automate the build and test processes (Singh et al. 2019). The main steps from one of the GitHub Actions workflow which shall be used, `dotnet.yml`, can be seen in **Figure 3.1**, and the associated code in **Listing 3.1**. This particular workflow is triggered when a developer creates a pull request and when a developer wishes to push code to the master branch. This workflow has one job titled `setup-build-test`, which completes steps in the following order:

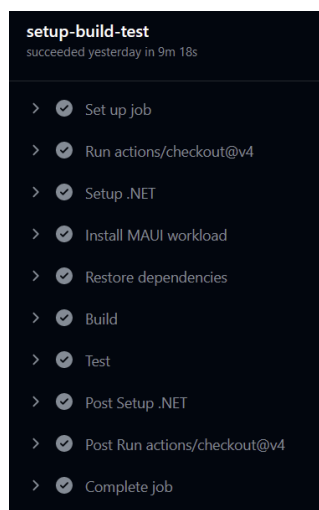


Figure 3.1: dotnet.yml GitHub Actions workflow

- **Setup .NET:** Sets up the necessary environment to compile the .NET code.
- **Install MAUI workload:** Installs the necessary libraries needed for compiling .NET MAUI applications.
- **Restore dependencies:** Downloads any additional dependencies which the .NET MAUI application needs to run.
- **Build:** Compiles the .NET MAUI code as would happen when running the artifact, Ensuring there are no errors when doing so.
- **Test:** Runs any automated tests associated with the code, which shall be discussed in the next chapter.

From **Listing 3.1**, the steps are clear and easy to read. Integrating this CI/CD pipeline helped ensure the artifacts code was free for errors, and subsequently helped create clean and functional code. For this artifact, code changes could only be merged once this workflow passed, acting as a safeguard to ensure the master branch was kept stable.

```
1 steps:
2 - uses: actions/checkout@v4
3 - name: Setup .NET
4   uses: actions/setup-dotnet@v4
5   with:
6     dotnet-version: 8.0.x
7
8 - name: Install MAUI workload
9   run: dotnet workload install maui
10
11 - name: Restore dependencies
12   run: dotnet restore
```

```
13
14 - name: Build
15   run: dotnet build --no-restore
16
17 - name: Test
18   run: dotnet test --no-build --verbosity normal
```

Listing 3.1: Main YAML file: `dotnet.yml`

In addition to the `dotnet.yml` workflow, a workflow titled `superlinter.yml` was used to scan the code for formatting issues. This proved to be useful as it ensured the C# code followed best practices before being pushed to the master GitHub branch.

To ensure the code had no security vulnerabilities, a `codeql.yml` workflow was created. CodeQL is an analysis engine, which is owned and thus supported by GitHub, which is used to automate security checks within the C# code of the artifact (GitHub 2024b).

3.2. Configuring the Solution

With the GitHub repository initialized, the implementation process can begin with the confidence that the artifact can be efficiently managed. This .NET MAUI solution was created in Visual Studio 2022, using the .NET 8 framework. From there it was linked to the GitHub repository which allowed commits to be made and managed with Visual Studio 2022. The artifact uses the MVVM pattern, which is common for .NET Maui applications and explained in further detail in the coming section.

3.2.1. Packages

NuGet package manager was used to handle any packages which were required. Those packages included:

- **HTMLAgilityPack** - Allows the C# code to scrape HTML information from the DoneDeal website; www.donedeal.com/cars.
- **CommunityToolkit.Mvvm** - Makes integrating the MVVM architecture a smooth and effective process, reducing the amount of code needed.

3.3. Code Structure

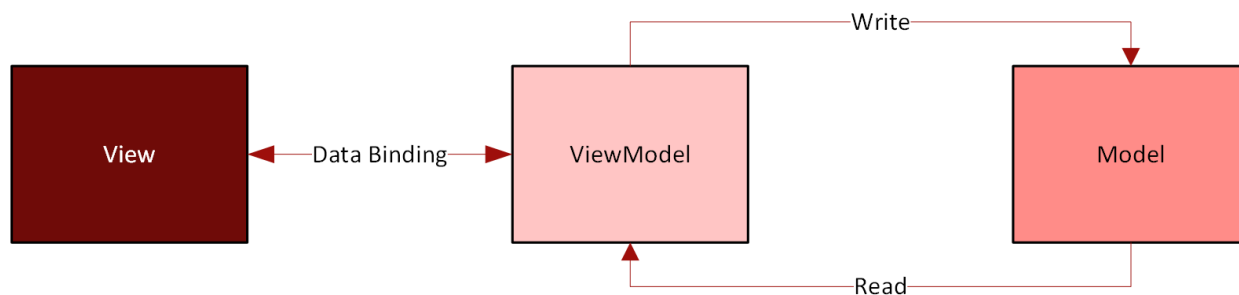


Figure 3.2: MVVM Structure

As mentioned, the code will follow the MVVM structure with the addition of a *Services* folder. The MVVM folder structure of the artifact can be visualized in **Figure 3.2** and is explained in detail below (TechTarget 2024):

- *Model* – Contains the data which the artifact will use.
- *ViewModel* – Implements commands between the View and the Model, essentially acting as a bridge between them.
- *View* – Is the user interface of the artifact which interacts with the ViewModel through data binding.

The *Services* folder contains the code necessary to send http requests to scrape information from the DoneDeal website. The ViewModel is the only content which interacts with the methods in the *Services* folder.

3.3.1. Models

The data classes for this project were each stored in the *Model* folder. In total there were three classes: *CarMake*, *CarModel* and *CarDetail*. *CarMake* stored only one string variable titled *Make* which represented the Make of car, for example Toyota, Ford. *CarModel* contained two string variables, *Make* and *Model* where *Model* references the Model of car, for example Toyota Corolla or Ford Focus. The final class, *CarDetail*, contained the car advertisement details, see the variables in bold with example values:

- **String Make** - Lexus
- **String Model** - RC
- **int Year** - 2018
- **String AdvertisementName** - Lexus RC300H F-Sport

- **double Price** - €30,000
- **String EngineSize** - 2.5 Petrol Hybrid
- **int Mileage** - 20,000 miles
- **String Location** - Co.Limerick
- **String URL** - www.donedeal.com/car-for-sale/lexus-rc300h-f-sport/12345678
- **bool IsHighlighted** - False

Whilst most variables are self explanatory, IsHighlighted perhaps is not. This variable becomes relevant when assessing if the specified car is under valued or not. In the scenario where the CarDetail object has a lower price and mileage than the average this variable is set to true, otherwise it is set to false. In the View section the reasoning for this is explained.

3.3.2. Views

The View folder contained the XAML files which this artifact used. Two XAML files were needed; MainPage and SummaryPage. The MainPage allowed the user to choose a specific car and the SummaryPage displayed the car information based on the user's selection. The overall design of both pages was kept minimalistic in order to make the artifact as user friendly as possible. As mentioned the data within these pages comes from the ViewModel which allows this XAML code to be as clean as possible.

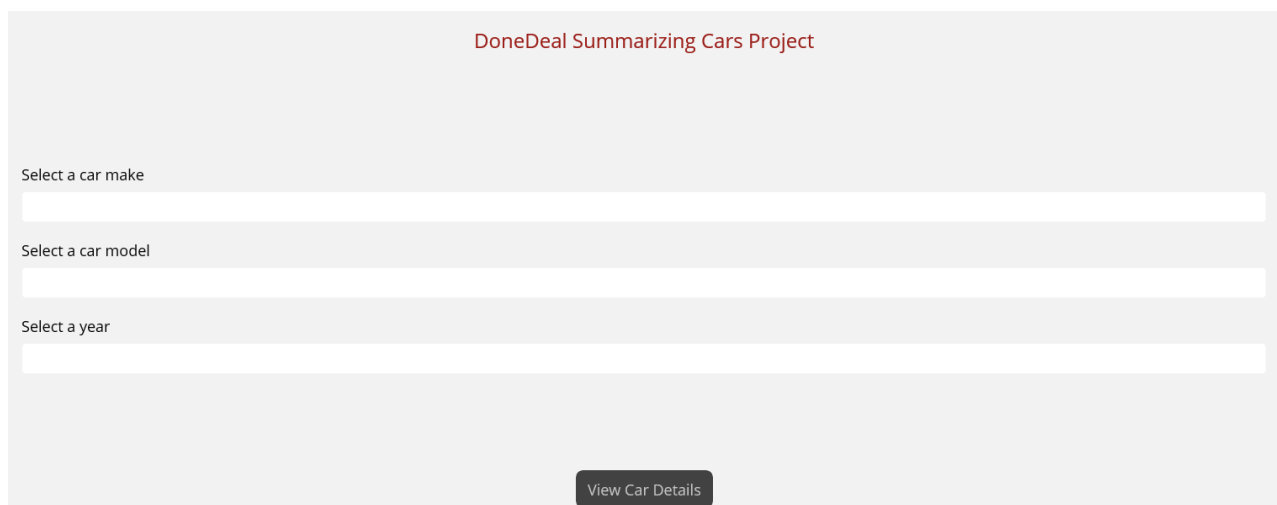
The image is a screenshot of a mobile application interface. At the top, there is a title bar with the text "DoneDeal Summarizing Cars Project" in a reddish-brown font. Below the title bar, there are three vertically stacked dropdown menus. The first is labeled "Select a car make", the second "Select a car model", and the third "Select a year". Each dropdown menu has a white background and a light gray border. At the bottom of the screen, there is a dark gray button with the text "View Car Details" in white.

Figure 3.3: MainPage of Artifact

The MainPage, as shown in **Figure 3.3**, has three drop downs which allow the user to select a car make, model, and year. Once the user has selected values for each of these drop downs the button to view car details becomes enabled. The drop down, or picker as they are denoted in XAML, for choosing a car make can be seen in **Listing 3.2**. What is notable about the XAML code for this picker is the keyword Binding. As demonstrated in **Figure 3.2**, the View and ViewModel are bound together through a process called data binding, as such any item with the key word Binding is linked

to the ViewModel. In this scenario the ItemSource is coming from a list called CarMakes which quite simply contains string variables of the available car makes. The SelectedItem is bound to a variable called SelectedCarMake which exists and is again stored in the ViewModel. Using this approach allows use to then use the car make the user has selected to obtain the appropriate car models in an effective manner. The other pickers follow a similar structure, with their data coming directly from the ViewModel.

```
1  <!-- Dropdown for selecting car make -->
2  <Picker ItemsSource="{Binding CarMakes}"
3      SelectedItem="{Binding SelectedCarMake, Mode=TwoWay}"
4      Title="Select a car make"
5      TitleColor="Black"
6      FontSize="18"
7      TextColor="#9E0F0B"
8      BackgroundColor="#FFFFFF"/>
```

Listing 3.2: XAML Drop Down

The button View Car Details allows the user to navigate to the summary page to see the details of their chosen car. From **Figure 3.3** the button is disabled as the user hasn't selected a car make, model and year using the pickers. Notice in **Listing 3.3** that IsEnabled is bound to a variable IsSummaryPageEnabled which is a boolean variable that is only true when the user has selected a car make, model and year. The Command of this button navigates to the SummaryPage, which is again handled by the ViewModel.

```
1  <!-- Button to navigate to SummaryPage -->
2  <Button Text="View Car Details"
3      Command="{Binding NavigateToSummaryPageCommand}"
4      IsEnabled="{Binding IsSummaryPageEnabled}"
5      HorizontalOptions="Center"
6      VerticalOptions="CenterAndExpand"
7      FontSize="18"
8      TextColor="#FFFFFF"
9      BackgroundColor="#9E0F0B"
10     Margin="0,20,0,0" />
```

Listing 3.3: XAML View Car Details Button

Once the user selects this button, they will be navigated to the SummaryPage as seen in . Notice in this scenario the user selected a 2017 Toyota Corolla.



Figure 3.4: SummaryPage of Artifact

In this page there are a few elements to note. The Select Engine Size drop down is simply a picker which has a code structure similar to **Listing 3.3**, with the data being bound from the SummaryPageViewModel. The three boxes which appear are each summary statistics of the chosen car. These values are the count of the cars visible, the average mileage and the average cost. Also visible is a collection showing each of the cars which fit the chosen criteria. The user can scroll through this collection to see the details of each car. Finally any car which is highlighted in light green should be noted as having a price which is less than the average price and a mileage which is less than the average mileage. As such this car may be of interest to a potential buyer as it may be undervalued.

Taking a look at the Average Mileage statistic, as shown in **Listing 3.4**, it is clear that the design is kept minimalistic. The AverageMileage value is coming from the ViewModel, and this value is formatted to zero decimal places. These summary elements each have a frame around them which helps make them stand out.

```

1  <!-- Average Mileage -->
2  <Frame Padding="10"
3      BackgroundColor="#F0F0F0"
4      CornerRadius="5">
5      <StackLayout>
6          <Label Text="Average Mileage:"
7              TextColor="Black"
8              FontSize="18"
9              HorizontalOptions="Center"/>
10         <Label Text="{Binding AverageMileage, StringFormat='{0:N0}}'"
11             TextColor="#9E0F0B"
12             FontSize="18"
13             FontAttributes="Bold"
14             HorizontalOptions="Center"/>

```

```

15     </StackLayout>
16 </Frame>

```

Listing 3.4: XAML Average Mileage

The final and perhaps most complex element was the `CollectionView` which displayed each car advertisement's details. One element which was crucial to include was the ability to highlight which cars were undervalued (which cars had below average price and mileage). **Listing 3.5** contains a snippet from this element which changes the frame color based on a boolean within `CarDetail` called `IsHighlighted`. In the scenario where the `CarDetail` object has this boolean set to true the frame color will be set to light green, otherwise it will be set to white.

```

1  <Frame BorderColor="LightGray" Padding="10" Margin="5">
2      <Frame.Triggers>
3          <DataTrigger TargetType="Frame" Binding="{Binding IsHighlighted}" Value="True">
4              <Setter Property="BackgroundColor" Value="LightGreen" />
5          </DataTrigger>
6          <DataTrigger TargetType="Frame" Binding="{Binding IsHighlighted}" Value="False">
7              <Setter Property="BackgroundColor" Value="#FFFFFF" />
8          </DataTrigger>
9      </Frame.Triggers>

```

Listing 3.5: XAML Highlighting Specific Car Advertisements

3.3.3. ViewModels

There were two classes in the `ViewModel` folder; `MainViewModel.cs` and `SummaryViewModel.cs`, each corresponding to `MainPage` and `SummaryPage`. Both of these classes contained the data which the Views used, and these classes also communicated with the methods stored in the `Services` folder.

Listing 3.6 showcases the constructor for the `MainViewModel` class. It firstly sets the `Title` variable, which simply appears at the top left of the associated View. This class initializes the services for fetching the car makes, models and years, and creates `ObservableCollections` to store this information. Note that these `ObservableCollections` were present in the `MainPage` View as they contained the data used for the drop downs. `NavigateToSummaryPageCommand` is an command which is triggered by the user hitting a button in the View. This command calls the `ExecuteNavigateToSummaryPageCommand` method, which navigates the user to the `SummaryPage` and creates a new instance of `SummaryViewModel` with the car input variables the user has selected. `LoadCarMakes` uses the `CarMakesService` to populate the `CarMakes` collection and `LoadCarYearRange` preforms a similar task with the `CarYearService`.

This class also stores additional properties bound to the `MainPage`, those being the car make, model and year that the user has selected. When navigating to the `SummaryPage` it will use these prop-

erties as input variables for the SummaryPage to display the chosen car advertisements. There is also a boolean variable, `IsSummaryPageEnabled`, which turns true only when the aforementioned properties have values assigned to them. This boolean, when true, allows the user to navigate to the SummaryPage.

```
1 public MainViewModel()  
2 {  
3     Title = "Front Page";  
4  
5     _carMakesService = new CarMakeService();  
6     _carModelService = new CarModelService();  
7     _carYearService = new CarYearService();  
8  
9     CarMakes = new ObservableCollection<string>();  
10    CarModels = new ObservableCollection<string>();  
11    YearRange = new ObservableCollection<int>();  
12  
13    NavigateToSummaryPageCommand = new Command(ExecuteNavigateToSummaryPageCommand);  
14  
15    LoadCarMakes();  
16    LoadCarYearRange();  
17 }
```

Listing 3.6: MainViewModel Constructor

The SummaryViewModel constructor code can be seen in **Figure 3.7**. It follows a similar structure to the MainViewModel with the addition of a string variable `SelectedCar` which is bound to, and appears at the top of, the SummaryPage. Notably this constructor contains a method `LoadCarDetailsAsync` which takes in the car make, model and year. This method uses the `CarDetailService` to get the car advertisements, adding them to the `CarDetails` collection. If this collection is empty it implies that no advertisements exist for the chosen car, and this navigates the user back to the MainPage. Otherwise it displays the car advertisements as shown in **Figure 3.4**.

Notable methods in this class are the `FilterCarDetails` and `UpdateAverages` methods. `FilterCarDetails` will filter the list of cars based on the engine size they choose in the SummaryPage dropdown. `UpdateAverages` is called right after the user filters by engine size, and this method calculates the average mileage and price values from the filtered list of cars the user has selected.

```
1 public SummaryViewModel(string selectedCarMake, string selectedCarModel, int selectedYear)
2 {
3     Title = "Summary Page";
4     SelectedCar = $"{selectedYear} {selectedCarMake} {selectedCarModel}";
5
6     _carDetailService = new CarDetailService();
7
8     CarDetails = new ObservableCollection<CarDetail>();
9     EngineSizes = new ObservableCollection<string>();
10
11     LoadCarDetailsAsync(selectedCarMake, selectedCarModel, selectedYear);
12 }
```

Listing 3.7: SummaryViewModel Constructor

3.3.4. Services

The most difficult part of the code to implement was the `Services` methods. This C# code was obtaining the information from the DoneDeal site, which involved scraping the data. Three classes were used; `CarMakeService`, `CarModelService`, `CarYearService` and `CarDetailService`. `CarDetailService` is used in the `SummaryViewModel`, and the remaining `Service` classes were used in the `MainViewModel`. `CarMakeService` and `CarYearService` were both structured in a very similar manner, `CarDetailService` followed a similar approach whilst being slightly more complex. `CarModelService` happens to obtain data from a CSV file due to the car models not being possible to scrape from the website, which is explained in the Conclusion chapter.

`CarMakeService` was the first `Service` class to be created, and its purpose is to scrape a list of available car makes from the DoneDeal site using the `GetCarMakesAsync` method. This was achieved using the `HTMLAgilityPack` library, which could send a GET request to `www.donedeal.ie/cars` and then parse the HTML response. **Listing 3.8** occurs only if the response is successful, and this code reads the HTML content as a string. Next, using `HTMLAgilityPack`, the code converts this string to a HTML item which can then be used to get the Car Makes. `doc.DocumentNode.SelectNodes` is used to find all of the Car Makes which are stored within HTML elements with class name `CustomSelect__ListItem-sc-1j4ks7m-2`. From there, provided at least one item is present, a list of `CarMake` objects are created and are returned for the `MainViewModel` to call and bind to the `MainPage`.

The `CarYearService` follows the exact same logic, with the exception being that it gets the to and from year values from the *from-year* drop down on the DoneDeal site and extracts every option (1980 until 2024). This uses the `GetYearRangeAsync` method to return two integer variables, those being the `FromYear` and `ToYear` which the `MainViewModel` uses to create a range of years between these two values.

```
1 HttpResponseMessage response = await client.GetAsync(URL);
2 if (response.IsSuccessStatusCode)
3 {
4     string HTMLContent = await response.Content.ReadAsStringAsync();
5     HTMLDocument doc = new HTMLDocument();
6     doc.LoadHTML(HTMLContent);
7
8     // HTML element which contains the dropdown items list of each car make
9     var dropdownItems = doc.DocumentNode.SelectNodes("//li[contains(@class, '
        CustomSelect__ListItem-sc-1j4ks7m-2')]//button//span");
10    if (dropdownItems != null)
11    {
12        var carMakes = dropdownItems.Select(item => new CarMake { Make = item.InnerText.Trim
            () }).ToList();
13        return carMakes;
14    }
15    else
16    {
17        Console.WriteLine("Dropdown items not found.");
18    }
19 }
```

Listing 3.8: CarMakeService Extracting Car Makes

The last Service class which the MainViewModel uses is the CarModelsService. Due to the structure of the DoneDeal site it was not possible to scrape the information as the items from the drop down for car models is not possible to access though sending a GET request. As such the CarModel objects were created by using a CSV file with two columns: Car Make and Car Model. The GetCarModelsAsync method used carMake as an input variable, which could be used to filter the CSV file to extract only the car models which were from the chosen car make. **Listing 3.9** showcases this method, which will return a list of CarModel objects if the user has selected a valid car make, otherwise it will return null and inform the user that their selected car make contained no car models.

```
1 public async Task<List<CarModel>> GetCarModelsAsync(string carMake)
2 {
3     // Read car models from CSV file
4     List<CarModel> carModels = await ReadCarModelsFromCSV();
5
6     // Filter car models based on car make
7     var filteredModels = carModels.Where(model => model.Make.ToLower() == carMake.ToLower()
8         ).ToList();
9
10    if (filteredModels.Any())
11    {
```

```
11     return filteredModels;
12 }
13 else
14 {
15     Console.WriteLine($"No car models found for '{carMake}'.");
16     return null;
17 }
18 }
```

Listing 3.9: CarModelService Extracting Car Models

The final Service method was CarDetailService, which was used by SummaryViewModel. This class used the GetCarDetailsAsync method which had three input variables; selectedCarMake, selectedCarModel and selectedYear. Due to DoneDeal having pagination, which essentially means the advertisements are stored on more than one HTML page, a while loop was used to scrape data from these pages. The URL variable, seen in **Listing 3.10**, deals with pagination using the ?start=X part of the URL. It should be noted that whilst traditional websites will use 1, 2, 3 ... for page numbers, DoneDeal uses the advertisement number to refer to the first page or first item that is shown, and it will show 30 of these advertisements per page. As such the start variable is initially 0 and increases by pageSize, which is set to 30, on every loop unless no more advertisements are present. the adContainers variable will contain a list of HTML nodes which correspond to an advertisement.

```
1 while (true)
2 {
3     string URL = $"https://www.donedeal.ie/cars/{selectedCarMake}/{selectedCarModel}/{
4         selectedYear}?start={start}";
5
6     HttpResponseMessage response = await client.GetAsync(URL);
7     if (response.IsSuccessStatusCode)
8     {
9         string htmlContent = await response.Content.ReadAsStringAsync();
10        HtmlDocument doc = new HtmlDocument();
11        doc.LoadHtml(htmlContent);
12
13        var adContainers = doc.DocumentNode.SelectNodes("//li[starts-with(@data-testid, '
14            listing-card-index')]");
15
16        ...
17    }
18
19    start += pageSize;
20 }
```

Listing 3.10: CarDetailService Pagination

This method will then extract the relevant CarDetail variables from each adContainer, whilst doing some basic transformations. For example for the EngineSize variable, some engine sizes appear as 1.8 Petrol Hybrid, or 1.8 Hybrid, or 1.8 Plug-in Hybrid, and each of these are in fact referring to the exact same 1.8 Hybrid engine. As such a method was created which finds any engine size which contains the word Hybrid and transforms that text to X Hybrid to keep the EngineSize variables consistent. Also for the Mileage variable, in some instances this appears in kilometers which is denoted by the keyword km. Thus a method was created which finds any mileage values with km and converts those to miles by multiplying by 0.621371.

4. Testing

To ensure the artifact functioned as expected, it was vital to test the functionality and user experience of the code. To test the functionality, the xUnit testing framework was used. xUnit allowed for the integration of unit tests, which is used to test the most granular elements of the code (Amazon Web Services 2024).

The user experience testing was less formal, as this focused more on the real world experience as opposed to the black and white functionality of the code. For this section user acceptance tests were performed, most of the steps involved in the acceptance testing took inspiration from the user stories.

4.1. Unit Tests

Unit testing proved to be the most efficient and effective method to test the code. The structure of these tests are consistent throughout: *Given*, *When*, *Then*. *Given* refers to the initial condition(s), *When* refers to the events or transformations which happen to the *Given* conditions, and *Then* is the expected result after the *When* step occurs (Ranorex 2023). To illustrate this method of testing, **Listing 4.1** below provides a basic example of a unit test built in C#.

```
1 public class ExampleUnitTest {  
2     [Fact]  
3     public void AddTwoToNumber_ShouldEqualThree()  
4     {  
5         // Given  
6         int n = 1;  
7  
8         // When  
9         int result = n + 2;  
10  
11        // Then  
12        Assert.Equal(3, result);  
13    }  
14 }
```

Listing 4.1: Example of a Basic Unit Test

The output from this test can be seen in **Figure 4.1** which shows that the test in **Listing 4.1** passed successfully. This figure is of the *Test Explorer*, which displays the output within Visual Studio 2022 of every unit test within the code grouped by class. These tests were carried out using the xUnit

framework, which was chosen due to its ease of integration. Setting up xUnit simply involved creating a new project within the artifact and selecting the "xUnit Test Project" template. From there the tests could be created as structured in **Listing 4.1**. Using this testing framework proved to be invaluable in ensuring the code functioned as expected, particularly as the running of these tests were automated with GitHub Actions.

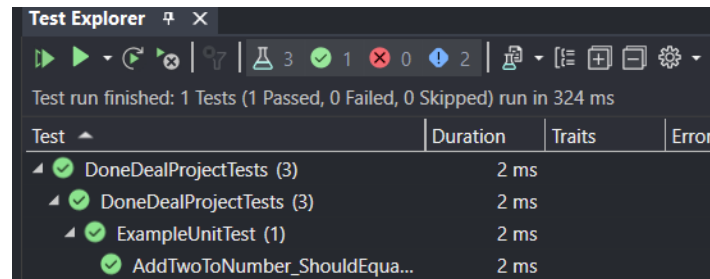


Figure 4.1: Test Explorer Output of a Basic Unit Test

4.1.1. Services Unit Tests

As the Services folder contained the more functional elements of the artifact's code, it was crucial to create unit tests for each method within this folder. Each of these service methods were tested in scenarios where they were expected to succeed and where they were expected to fail. The output from the Test Explorer can be seen in **Figure 4.2**

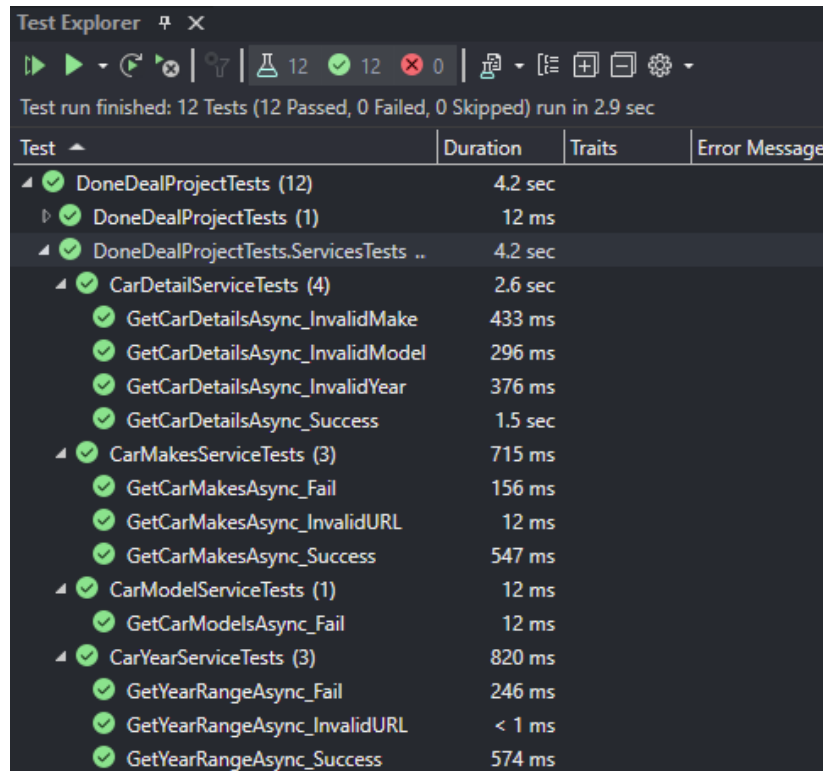


Figure 4.2: Test Explorer Output for the Artifact's C# Code

The tests which were carried out can be seen below, each has a unique identifier **Table 4.X**, where **X** is the test number.

CarMakes - GetCarMakesAsync_Fail
This test takes an unexpected URL input for the GetCarMakesAsync method and should return null. The expected input is the www.donedeal.ie , and the unexpected input which shall be used is the site www.youtube.com .
Given CarMakesService object
When creating a list of CarMakes using an incorrect URL input www.youtube.com .
Then the output should be a null list.
Success

Table 4.1: CarMakes - GetCarMakesAsync_Fail

CarMakes - GetCarMakesAsync_Success
This test takes the expected input for the GetCarMakesAsync method and should return the list of car makes. The expected input is the DoneDeal website, which happens to be the default input as the input for this method is optional. The expected output will be a list of all available car makes from the DoneDeal site.
Given CarMakesService object
When creating a list of CarMakes using the default URL input, which is the DoneDeal site
Then the output should be a a list of all available car makes from DoneDeal site.
Success

Table 4.2: CarMakes - GetCarMakesAsync_Success

CarMakes - GetCarMakesAsync_InvalidURL
This test takes an unexpected input which is not a valid URL for the GetCarMakesAsync method and should return null. The expected input is the www.donedeal.ie , and the unexpected input which shall be used is <i>Not a valid URL</i> .
Given CarMakesService object
When creating a list of CarMakes using an incorrect input which is not a valid URL
Then the expected result is to throw an InvalidOperationException.
Success

Table 4.3: CarMakes - GetCarMakesAsync_InvalidURL

GetCarModelsAsync_Fail
This test takes an unexpected input for the GetCarModelsAsync method and should null. In this case an unexpected input is a car make which is not present on DoneDeal, as such the input will be the string <i>Not A Car Make</i> .
Given CarModelService object
When creating a list of CarModels using an incorrect input such as <i>Not A Car Make</i>
Then the output should be null.
Success

Table 4.4: GetCarModelsAsync_Fail

5. Conclusion

The final artifact has met expectations and provides the user with the necessary car summary information. The user is able to select a car make, model and year, and upon a valid selection can see a list of all car advertisements that meet that criteria. The user can also filter by engine size and see the average mileage and average cost of their selection. The Design chapter of the artifact describes the necessary requirements of the artifact from multiple perspectives, all of which were incorporated in the final solution. This chapter also used diagrams to describe how the artifact should function, which fed into the Implementation chapter. This chapter focused on the main components of the artifact's code, whilst also giving some more context on how GitHub was used to implement CI/CD principles. Finally the Testing chapter tied both the Design and Implementation chapters together by performing the relevant tests to confirm that the artifacts code worked as expected and that the user experience was as expected.

In terms of future development, it would be interesting to create a predictive model to calculate the predicted price of a certain car based on it's mileage, engine size, year and perhaps other variables such as features.

5.1. Challenges

The first challenge when it came to this project was figuring out the best method and best framework to use for testing. After considering the different options it became clear that xUnit was the best choice. This was due to the ease of integration and abundance of documentation material online, in addition to the Test Explorer window which allowed tests to be effectively managed.

Perhaps the biggest challenge was figuring out how to get the car model information based on the car make the user selected. Initially it seemed like the obvious approach would have been to use DoneDeal site with the HtmlAgilityPack package to scrape the data, but after testing it became obvious that this would not work. After selecting a car make, such as BMW, the DoneDeal URL updates from `www.donedeal.com/cars` to `www.donedeal.com/cars/bmw`. Notably, right after you select BMW the drop down for the BMW models appears, allowing you to select a specific model. The issue occurs after navigating to the same URL, `www.donedeal.com/cars/bmw`, which mimics a user selecting BMW from the make drop down. When a user does this the model drop down seems to disappear, and the car make drop down changes to BMW All Models, which doesn't allow for the model to change dynamically. As such the solution was to create a CSV file to contain a mapping of car make to car model. Whilst this approach was not the preferred method, it was one of the only methods which would work in the interim.

The next challenge came from getting all of the car information due to pagination. With most web-

sites, the pages are numeric, starting from either 0 or 1 and increasing in increments of 1. For the DoneDeal website however the pages worked based on advertisement number. As such the URL query `?start=X` was used instead of `?page=X`. This start query began at 0 and was iterated by 30 to go to the next page, as each page contained 30 advertisements.

A challenge which only occurred in testing was the `GerCarModelsAsync_Pass` test which is meant to read a CSV to create `CarModel` objects corresponding to the chosen car make. Ultimately this test was removed due to the test solution seemingly being unable to access the associated CSV file. After hours of trying to find a solution it was deemed logical to omit this test.

5.2. Learning Outcomes

The main learning outcome from this project was how to effectively manage the development of a .Net MAUI artifact. One aspect which was particularly useful was the incorporation of GitHub Actions, which allowed for CI/CD workflows. These workflows were triggered on pull requests to the master GitHub branch, which allowed for the code to be built, tested, checked for code styling issues and scanned for security flaws. Whilst it added more time to the development process, most notably at the beginning, it proved to be incredibly useful for keeping the code as clean and consistent as possible.

Another learning outcome was the importance of creating versions of the artifact. This was easily managed with GitHub, and allowed for developments to be made to the code without directly modifying the master branch of the code. In essence this meant it was possible to experiment with different ideas on how the final product should behave.

References

- Amazon Web Services (2024). *Unit Testing*. URL: <https://aws.amazon.com/what-is/unit-testing/>.
- Atlassian (2024). *User Stories: Definition and Examples*. URL: <https://www.atlassian.com/agile/project-management/user-stories>.
- AutoTrader PLC (2024). *Retail Price Index*. URL: <https://plc.autotrader.co.uk/news-views/retail-price-index/>.
- DoneDeal (2024). *DoneDeal Q4 2023 and 2024 Outlook: Motor Report - Era of Pandemic-Induced Inflation Begins to Subside*. URL: <https://www.blog.donedeal.ie/post/donedeal-q4-2023-and-2024-outlook-motor-report-era-of-pandemic-induced-inflation-begins-to-subside>.
- Gall, Michael and Federico Pigni (2022). "Taking DevOps mainstream: a critical review and conceptual framework". In: *European Journal of Information Systems* 31.5, pp. 548–567. URL: <https://www.tandfonline.com/doi/abs/10.1080/0960085X.2021.1997100>.
- GitHub (2024a). *About GitHub and Git*. URL: <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>.
- (2024b). *CodeQL Documentation*. URL: <https://codeql.github.com/docs/>.
- Independent.ie (2023). *Prices of used cars still almost double the level before pandemic*. URL: <https://www.independent.ie/irish-news/prices-of-used-cars-still-almost-double-the-level-before-pandemic/a2037577377.html>.
- Irish Examiner (2024). *Used car prices still up 85% despite 'considerable ease' in inflation*. URL: <https://www.irishexaminer.com/business/economy/arid-41326783.html>.
- Jarzębowicz, Aleksander and Paweł Weichbroth (2021). "A qualitative study on non-functional requirements in agile software development". In: *IEEE Access* 9, pp. 40458–40475.
- Malan, Ruth, Dana Bredemeyer, et al. (2001). "Functional requirements and use cases". In: *Bredemeyer Consulting*, pp. 335–1653.
- Microsoft (2023). *What is .NET MAUI*. URL: <https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui?view=net-maui-8.0>.
- Raharjana, Indra Kharisma, Daniel Siahaan, and Chastine Fatichah (2021). "User stories and natural language processing: A systematic literature review". In: *IEEE access* 9, pp. 53811–53826.
- Ranorex (2023). *Given-When-Then Tests*. URL: <https://www.ranorex.com/blog/given-when-then-tests/>.
- Singh, Charanjot et al. (2019). "Comparison of different CI/CD tools integrated with cloud platform". In: *2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE, pp. 7–12.

- Statista (2020). *Impact of COVID-19 on the Automotive Industry Worldwide*. URL: <https://www.statista.com/topics/8749/impact-of-covid-19-on-the-automotive-industry-worldwide/>.
- TechTarget (2024). *Model-View-ViewModel*. URL: <https://www.techtarget.com/whatis/definition/Model-View-ViewModel>.
- Xinogalos, Stelios (2013). "Using flowchart-based programming environments for simplifying programming and software engineering processes". In: *2013 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, pp. 1313–1322.